

CMPE-679 Lab 2

TA: Ishan Renjen

February 12, 2026

[Lab 2 Paper](#)
[Lab 2 Dataset](#)

1 Introduction

This lab is implementing the Transformer paper, Attention is All You Need. This was the first paper introducing the Transformer which has changed the course of ML. In contrast to recent transformer architectures which use just a decoder, the transformer you will create uses an encoder and decoder. Encoder-decoder architectures are more complex than decoder-only architectures, but are useful to understand transformers and a lot of ML concepts such as attention and writing PyTorch code for larger, more complex models.

In this lab, you will write the code for a transformer. First, read the paper in detail and understand how dot-product attention, positional encodings, cross-attention, and masking works for this model. Then, implement the exact model shown in the paper. Be sure to pay attention to the details such as the 6 encoder and decoder layers, the dimensions and the dropout level. Points will be deducted for deviations from the paper in your code.

For evaluation, ideally use the train/test/validation loss for english to german. Any language can be picked, but the original paper uses english to german. The requirement is 10 translations - english - german/any language - english. The translation back to english must be relatively correct in comparison to the input. **This should be a screenshot, and I need to see this in the output of the Jupyter Notebook cell.**

2 Deliverables

Google Drive link to a folder with:

- Jupyter Notebook (.ipynb) with all the code for the assignment - **The code should be fully functional (run a full epoch of training/eval or no points will be given).**
- Model file (.pth) saved from however much training is done (**minimum 1 epoch**)
- Short informal report with lab description, results, issues encountered, training loss, testing loss, validation loss, translations, and potential improvements - **The report must be included or no points will be given.**

3 Grading

- **Functional Jupyter Notebook and Model:** 30 pts
- **Accurate implementation of paper:** 30 pts
- **10 coherent translations:** 20 pts
- **Report:** 10 points
 - Summary of lab (explain purpose of lab, data, model): 5 points

- Training, Testing, and Validation loss over all epochs trained: 10 points
- Conclusion (explain issues, potential improvements, etc): 5 points
- **Questions in Report:** 10 points
 - Q1:** Feedback on lab: anything you would like to see changed: 2 points
 - Q2:** Explain how Cross-Attention works and why it is used in this model: 5 points
 - Q3:** Explain what the difference is in an encoder-decoder transformer vs. just a decoder: 3 pts
- **Bonus:** 20 pts
 - : Complete 10+ epochs of training using model checkpointing with [PyTorch Lightning](#) and give a BLEU score - there are a lot of functions to do this

4 Starter Code:

Some code is provided below to make dataloading easier, [make sure to read the documentation for tokenizer and DataCollatorForSeq2Seq](#).

```
import numpy as np
import matplotlib as plt
import torch
import torch.nn.functional as f
from datasets import load_dataset
import os
import math
import random

from transformers import DataCollatorForSeq2Seq
from torch.utils.data import DataLoader
from tokenizers import Tokenizer
from tokenizers.models import BPE
from tokenizers.trainers import BpeTrainer
from tokenizers.pre_tokenizers import Whitespace
from transformers import PreTrainedTokenizerFast
from google.colab import drive
drive.mount('/content/drive')

#https://huggingface.co/datasets/wmt/wmt14
ds = load_dataset("wmt/wmt14", "de-en")

train_ds = ds["train"]
test_ds = ds["test"]
val_ds = ds["validation"]

tokenizer = None
tokenizer_path =
    ↪ "/content/drive/MyDrive/CMPE-679-Labs/Lab2-Transformer/my_trained_tokenizer.json"
    ↪ #this path should be different for you

if not os.path.exists(tokenizer_path):
    #https://huggingface.co/docs/transformers/en/fast_tokenizers
    tokenizer = PreTrainedTokenizerFast(BPE(unk_token="[UNK]"))
    trainer = BpeTrainer(vocab_size=37000,
                        special_tokens=["[PAD]", "[UNK]", "[BOS]", "[EOS]"])
    tokenizer.pre_tokenizer = Whitespace()

def tokenizer_generator(ds):
    for row in ds:
        yield row["translation"]["en"]
        yield row["translation"]["de"]
```

```

tokenizer.train_from_iterator(tokenizer_generator(ds['train']), trainer)
tokenizer.save(tokenizer_path)
else:
tokenizer = PreTrainedTokenizerFast(tokenizer_file=tokenizer_path)
tokenizer.pad_token = "[PAD]"
tokenizer.bos_token = "[BOS]"
tokenizer.eos_token = "[EOS]"

max_len = 128
batch_size = 128

def preprocess(batch):
    src = batch["translation"]["de"]
    tgt = batch["translation"]["en"]

    model_inputs = tokenizer(src, max_length=max_len, truncation=True)
    labels = tokenizer(text_target=tgt, max_length=max_len, truncation=True)

    model_inputs["labels"] = labels["input_ids"]
    return model_inputs

#https://huggingface.co/docs/datasets/en/use_dataset
train_tokens = ds["train"].map(preprocess, remove_columns=ds["train"].column_names,
    ↪ num_proc=4)
test_tokens = ds["test"].map(preprocess, remove_columns=ds["test"].column_names, num_proc=4)
val_tokens = ds["validation"].map(preprocess,
    ↪ remove_columns=ds["validation"].column_names, num_proc=4)

collator = DataCollatorForSeq2Seq(tokenizer=tokenizer, padding=True,
    ↪ label_pad_token_id=tokenizer.pad_token_id)

train_loader = DataLoader(train_tokens, batch_size=batch_size, shuffle=True,
    ↪ collate_fn=collator)
test_loader = DataLoader(test_tokens, batch_size=batch_size, shuffle=True,
    ↪ collate_fn=collator)
val_loader = DataLoader(val_tokens, batch_size=batch_size, shuffle=False,
    ↪ collate_fn=collator)

```